

Ankur Desai — Bullet Line Test

All canonical bullets from context.md rendered with the resume template

EXPERIENCE

Secure and Efficient Autonomous Systems Lab

May 2026 – Present

Undergraduate Research Assistant

East Lansing, MI

- Researching the security of an LLM-controlled robot, where the cheap per-step gate screening its motion plans is the exact seam an attacker slips a harmful plan through, one innocent fragment at a time.
- Reframed the contribution from building a defense to measuring one, arguing that a triage gate's cost savings and its attackability reduce to a single property: per-step versus whole-plan checking.
- Ran the node graph byte-identical in Gazebo and on a physical RoboMaster EP Core, driving the real robot over a custom C++ WiFi driver at a 30 Hz control loop with sub-20ms command latency.
- Built the verifier as a C++ ROS2 node running per-step geometric and semantic checks on each motion plan, integrating nav2's MPPI controller rather than hand-rolling one to keep the C++ work on security, not motion control.
- Reproduced a published adversarial-robotics attack loop (attacker, target, judge, syntax-checker) as the baseline, then drove the decomposition attack to an 80% success rate against the per-step triage gate.
- Built the attacked defense as a steelman composed from multiple published safety architectures rather than a naive single classifier, so the decomposition attack lands on the strongest reasonable system, not a strawman.

Claude Builder Club @ MSU

Oct 2025 – Present

Co-Founder & Vice President

East Lansing, MI

- Co-founded and grew an MSU engineering community from zero to 150+ members, replacing a sprawl of spreadsheets, Slack threads, and manual invites with a single club-operations platform I designed and built.
- Automated member onboarding so an accepted applicant is provisioned into the right GitHub teams, Slack channels, Discord roles, and Google Drive access in a single flow, cutting onboarding from 15 minutes to seconds.
- Built event check-in as a single atomic database RPC that validates a QR token, records attendance, and awards points server-side, making door-time check-ins concurrency-safe and idempotent, sustaining 150+ check-ins per event.
- Built the club's mentorship lineage as an interactive 3D radial graph in react-three-fiber, hand-writing a proportional-arc layout that prevents edge crossings and a custom camera that autofits each family to the viewport.
- Pushed authorization into the database with row-level security so every table enforces access in Postgres regardless of what the client sends, letting new features ship without re-implementing per-endpoint auth, and scoped realtime subscriptions to that same filter so a member sees their own role or acceptance change live.
- Built a hands-off attendance-enforcement pipeline that runs a daily in-database cron job, resolves absentees, and posts targeted Slack reminders, automating a three-strikes rule that previously required manual tracking.
- Shipped the platform continuously from main on a GitHub Actions pipeline that applies database migrations and deploys edge functions on every push, taking the team from manual changes to reviewed continuous deploys.
- Tuned React Query cache tiers by data volatility across the dashboards, caching stable roles longer than fast-changing application queues to cut redundant refetches while keeping live data fresh.

MindMosaic

Aug 2025 – May 2026

Software Engineering Intern

East Lansing, MI

- Built and shipped an interactive knowledge-graph platform that keeps dense, interconnected content navigable as it grows, delivering a production MVP to real users on a polyglot Spring Boot, Neo4j, and PostgreSQL stack.
- Migrated hot read paths off recursive SQL self-joins onto a Neo4j graph layer so relationships stay first-class, cutting query latency 48% (850ms to 440ms) and making multi-hop traversals 3x faster than the equivalent SQL.
- Designed a dual-database layer that keeps transactional records in PostgreSQL while serving relationship-heavy reads from Neo4j, routing each query class to the store that answers it fastest instead of forcing one engine to do both jobs.
- Owned the Spring Boot backend serving the platform's core CRUD and graph-traversal APIs, structuring endpoints around the dual-store model so the client never has to know which database backs a given read.
- Authored an AWS deployment script and managed the environment as code with Terraform, replacing a manual 30-minute multi-service bring-up with a single-command, version-controlled provision.

Robostangs (FRC Team 548)

Jun 2023 – May 2024

Software Captain

Northville, MI

- Led the software and electrical teams (5 and 6 engineers) for a competition robot that reached the FIRST World Championship and placed top 16 in its division in Houston, the program's strongest world-stage finish since 2014.

- Delivered the team's first multi-piece autonomous routine, chaining vision-corrected driving paths with timed intake-and-shoot actions so the robot scored multiple game pieces with no driver input during the autonomous period.
- Anchored auto-aim on real-time, vision-corrected localization that fused two AprilTag cameras with swerve odometry on a dedicated 250Hz thread with latency compensation, holding pose accurate during aggressive motion.
- Hardened pose estimation against bad vision data by gating camera measurements on tag count, target area, and angular velocity before fusing them, rejecting estimates that would otherwise poison the localization filter.
- Engineered graceful sensor-failure handling so the most match-critical mechanism falls back to an internal encoder and stays operable through a sensor loss instead of bricking mid-match, surfaced as an alert rather than a silent failure.
- Built shoot-from-anywhere aiming that maps target distance to arm angle through a regression model, keeping candidate models swappable behind one interface so the team could A/B them against measured shot data.

PROJECTS

Dadei | *Line test*

dadei.app

Ambient voice assistant that turns overheard conversation into human-approved calendar, email, and task actions

- Routed all model-proposed side effects through one approval chokepoint with a cancelable countdown, giving every irreversible action one auditable seam between the LLM and the outside world.
- Built a React client that streams microphone audio over a WebSocket with client-side Web Audio processing and wires a countdown banner to the action queue, rendering live captions in under 300ms and letting the user cancel the on-screen timer to abort a queued Redis job before it auto-fires and promote the next pending action.
- Decoupled background workers from the live API behind a Redis Streams event bus with consumer groups, dead-letter routing after 5 retries, and stale-claim recovery, giving at-least-once delivery that Redis pub/sub dropped.
- Engineered a no-merge-first speaker-ID model using an EMA voice centroid plus a bounded prototype bank with floor-and-margin gating, abstaining into a new identity on ambiguity, attributing 96% of utterances to the correct speaker.
- Trained a custom wake-word model to 90% recall at under one false accept per hour, gating the command pipeline so the assistant only opens the mic on demand instead of continuously streaming audio.
- Solved the same-room, multi-mic problem with two arbitration layers, a live utterance-owner election and a 400ms windowed quality election on SNR, clarity, and cadence, cutting duplicate transcription by 50%.
- Split speech recognition into a Deepgram streaming path for live command latency and a batched Whisper path for ambient bulk transcription, keeping interactive commands responsive while bulk transcription stays cheap.
- Replaced single-signal context lookup with hybrid retrieval over pgvector that fuses vector similarity, lexical overlap, recency decay, and participant overlap, grounding each proposal under a fixed token budget.
- Shipped the backend as two services from one Docker image by role, with boot-time migrations and a three-layer restart strategy surviving the worker queue's silent exit-zero death on a Redis drop.

fliks | *Line test*

github.com/fliks-gg

Multi-game platform that crowd-validates gameplay skill and mints shareable certification cards

- Chose a single Go monolith over a microservice split, keeping auth, rate-limiting, and rating aggregation in-process because premature decomposition is the harder call to defend at pre-scale, one deploy unit against one datastore.
- Built a self-hosted video transcoding pipeline with a concurrent Go worker that pulls upload jobs, transcodes, and writes processed video back to object storage, owning the media path end to end instead of renting a managed service.
- Coordinated transcode jobs through a Postgres-backed queue using SELECT FOR UPDATE SKIP LOCKED instead of bolting on Redis or SQS, keeping the whole system on one datastore and one self-provisioned EC2 box.
- Architected an ML rating service that grades each clip independently and shows its verdict beside the crowd's, with those crowdsourced ratings serving as the labeled data the classifier trains on.
- Made ratings immutable and gated commenting behind a submitted rating, so the certification signal cannot be retroactively gamed and social participation feeds skill data instead of diluting it.

WizViz | *Line test*

devpost.com/software/wizviz

Webcam fighting game driven by real-time body-gesture recognition, winner of the Interactive Media track at SpartaHack X

- Stabilized noisy per-frame pose output by accumulating every frame's gesture classification across the turn window and taking the dominant sustained gesture, instead of reading a single jittery frame at the turn boundary.
- Classified five distinct combat gestures from body landmarks with a scale-invariant geometric model normalized by torso length, making detection work across players and camera distances without any labeled training data.
- Ran pose inference asynchronously off the render loop with a monotonic timestamp guard that drops out-of-order results, sustaining 60 FPS with sub-20ms input lag instead of blocking on per-frame detection.
- Supported both local two-player, splitting players by landmark position across the frame with zero extra cameras, and single-player against a finite-state opponent that rests, defends, heals, or finishes by game state.